



南開大學
Nankai University

计 算 机 学 院

大数据计算及应用实验报告

期末作业
Recommendation Systems

姓名：胡进喆 原敬闰 张明昆

学号：2213045 2211771 2211585

专业：计算机科学与技术 物联网工程

指导老师：杨征路

日期：2025 年 6 月 5 日

摘要

目录

- 1. 实验背景
- 2. 实验要求
- 3. 数据集说明
 - 3. 1. train.txt
 - 3. 2. test.txt
- 4. 协同过滤基础算法
 - 4. 1. User-Based CF
 - 4. 1. 1. 算法原理
 - 4. 1. 2. 代码实现
 - 4. 2. Item-Based CF
 - 4. 2. 1. 算法原理
 - 4. 2. 2. 代码实现
 - 4. 3. Slope one
 - 4. 3. 1. 算法原理
 - 4. 3. 2. 代码实现
- 5. 图模型与集成方法
 - 5. 1. Graph CF
 - 5. 1. 1. 算法原理
 - 5. 1. 2. 代码实现
 - 5. 2. TopKNanCF
 - 5. 2. 1. 算法原理
 - 5. 2. 2. 代码实现
- 6. 线性优化模型
 - 6. 1. GDLinearCF
 - 6. 1. 1. 算法原理
 - 6. 1. 2. 代码实现
 - 6. 2. LeastSquares CF
 - 6. 2. 1. 算法原理
 - 6. 2. 2. 代码实现
- 7. 矩阵分解模型
 - 7. 1. SVD

- 7. 1. 1. 算法原理
 - 7. 1. 2. 代码实现
- 7. 2. SVD++
 - 7. 2. 1. 算法原理
 - 7. 2. 2. 代码实现
- 7. 3. NMF
 - 7. 3. 1. 算法原理
 - 7. 3. 2. 代码实现
- 8. 实验结果
 - 8. 1. 时间与内存消耗分析
 - 8. 2. 全局误差对比
 - 8. 3. 模型特性分析
 - 8. 3. 1. 基于不同邻居数量下的误差对比
 - 8. 3. 2. 基于GraphCF的不同参数分析

1. 实验背景

2. 实验要求

3. 数据集说明

我们编写 `data_analysis.py` 对数据集进行统计分析，得到如下结果：

3.1 train.txt

数据集概览为：

属性	数值	属性	数值
评分数量	90,854	用户数量	598
用户平均评分数量	151.93	物品数量	9,077
物品平均评分数量	10.01	用户ID范围	1~610
评分均值	69.88	数据集稀疏度	98.33%
评分标准差	20.78	物品ID范围	1~193609

从用户 ID 范围来看，最小用户 ID 为 1，最大用户 ID 为 610，表明用户 ID 是连续且紧密分布的。物品 ID 范围从 1 到 193609，呈现出较大的跨度，与物品数量（9077）对比，说明物品 ID 存在明显不连续的情况。每个用户的平均评分数量为 151.93 条，表明数据集中用户参与评分的活跃度较高。反观物品维度，平均每个物品仅获得约 10.01 个评分。这揭示了物品的受欢迎程度分布不均，多数物品可能仅获得少数用户关注，长尾效应显著，部分冷门物品的评分稀缺可能影响推荐算法对这些物品的准确评估。

评分的均值为 69.88，标准差为 20.78，表明评分数据整体上呈现出较为集中的趋势，但也有一定程度的波动。从分位数来看，中位数为 70，25% 分位数为 60，75% 分位数为 80。这暗示着评分分布可能呈现出一定的正偏态，即多数评分集中在中高分段，低分段的评分数量相对较少。

评分稀疏度高达 0.9833，即数据矩阵中约 98.33% 的位置是空缺的，这在推荐系统中属于非常高的稀疏度水平，给模型训练带来了极大的挑战，需要采用有效的矩阵填充或降维方法。

下表展示了各评分值的出现频率：

评分值	出现次数	评分值	出现次数
10	1221	60	18119
20	2552	70	11996
30	1603	80	24177
40	6852	90	7742
50	5067	100	11525

从上表可以看出，高分值（如 60、80、100）的出现频率明显高于低分值。评分 80 出现次数最多，达到 24177 次，占总评分数的约 26.6%，而评分 10 的出现次数最少，仅占约 1.34%。这种评分分布表明用户在评分时更倾向于给出中高分评价，可能反映出数据集整体质量较高，或者存在用户评分偏乐观的倾向。

💡 Tip

在实际实验中，由于test.txt仅包含用户-物品对而未给出真实评分，无法直接用于模型误差的评估，因此本小组采用“交叉验证”或“留出法”将原始数据集划分为训练集和验证集。具体做法是：首先将完整的评分数据（train.txt）按一定比例（8:2）划分为训练集和验证集，训练集用于模型参数学习，验证集则保留真实评分用于评估模型的预测能力。通过在验证集上计算MAE、RMSE等指标，可以客观反映模型在未见数据上的泛化性能。

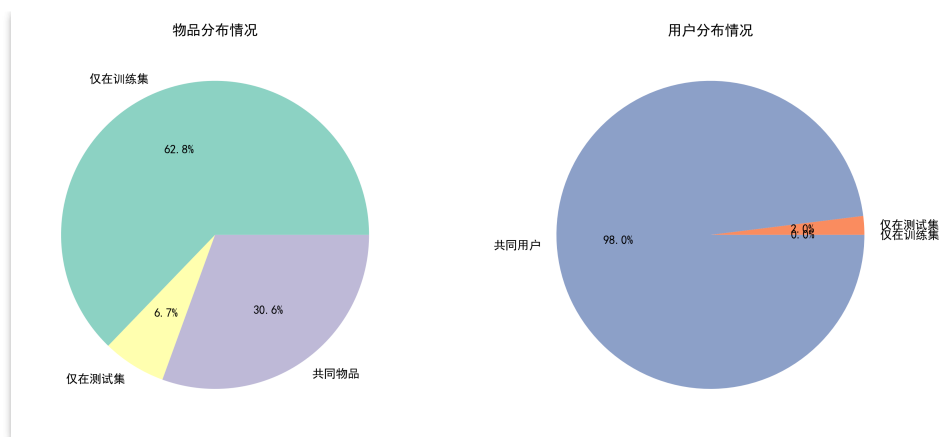
3.2 test.txt

同样，对测试集进行分析得到如下结果：

属性	数值	属性	数值
评分数量	9982	用户ID范围	1~610
用户数量	610	物品ID范围	1~193565
物品数量	3618		

从数据规模上来看，test集覆盖了完整的用户ID范围（1-610），但包含3618个物品，仅占完整物品ID范围（1-193565）的约1.87%，这表明测试集在物品维度上具有较高的稀疏性，物品覆盖率较低，这反映了推荐系统在实际应用中面临的冷启动问题。

因此我们对训练集和测试集进行冷处理分析，得到如下图表：



上述饼图直观展示了训练集与测试集在物品和用户分布上的差异：左图显示，约62.8%的物品仅出现在训练集，30.6%的物品为训练集和测试集共有，仅有6.7%的物品为测试集独有，说明测试集存在比例为17.88%的冷启动物品；右图则表明，绝大多数用户（98.03%）在训练集和测试集中均有出现，只有极少数用户为测试集独有，用户冷启动问题并不突出。整体来看，**推荐系统在物品维度面临更为显著的冷启动挑战，而用户维度的数据分布则较为充分。**

Important

通用技术规范： 后续算法实现遵循统一技术标准，建立"复用"标记体系，形成技术继承关系，进而增强实验的连贯性： 1.**相似度计算**：默认使用皮尔逊相关系数 2.**冷启动处理**：新用户/新物品返回全局平均分 3.**评估指标**：采用MAE和RMSE双指标 4.**矩阵存储**：使用稀疏矩阵格式CSR

4. 协同过滤基础算法

4.1 User-Based CF

User-Based协同过滤（User-Based Collaborative Filtering，简称User-Based CF）是一种基于群体智慧的核心推荐方法，其思想源于"**相似用户可能对未交互项目具有相近的偏好**"。该方法通过分析用户历史行为数据，挖掘用户间的相似性关系，并利用相似用户对目标项目的评分来预测目标用户的潜在兴趣。

4.1.1 算法原理

User-Based CF的输入依赖于用户对项目的显式或隐式反馈数据。显式反馈（主动评分）包括用户直接对商品、电影等项目的评分或评价，而隐式反馈（被动评分）则通过用户行为（如购买记录、浏览时长、点击率）间接反映兴趣强度。例如，电子商务场景中，用户的购买行为天然构成隐式评分矩阵，其中购买频次或金额可量化为评分值。这些数据被组织为**用户-项目评分矩阵**，矩阵中的**每个元素 $R_{u,i}$ 表示用户 u 对项目 i 的评分**，未评分的项目则作为待预测目标。

相似度计算 核心假设是相似用户对同一项目的评分具有一致性。为此，需定义用户间相似性度量方法，常见算法包括：

皮尔逊相关系数： 衡量两位用户评分趋势的线性相关性，通过消除用户评分尺度偏差（如部分用户习惯性打高分或低分）提升相似度准确性。公式为：

$$\sin(u, v) = \frac{\sum_{i \in I_{uv}} (R_{u,i} - \bar{R}_u)(R_{v,i} - \bar{R}_v)}{\sqrt{\sum_{i \in I_{uv}} (R_{u,i} - \bar{R}_u)^2} \sqrt{\sum_{i \in I_{uv}} (R_{v,i} - \bar{R}_v)^2}}$$

- 其中， I_{uv} 为用户 u 与 v 共同评分的项目集合， R_u 、 R_v 为各自的平均评分。皮尔逊系数范围在 $[-1, 1]$ ，值越大表示用户偏好越相似。

余弦相似度： 将用户评分视作向量，计算其夹角的余弦值以衡量方向相似性，适合处理稀疏数据。公式为：

$$\sin(u, v) = \frac{\sum_{i \in I_{uv}} R_{u,i} \cdot R_{v,i}}{\sqrt{\sum_{i \in I_{uv}} R_{u,i}^2} \cdot \sqrt{\sum_{i \in I_{uv}} R_{v,i}^2}}$$

- 调整后的余弦相似度进一步考虑项目平均评分，消除热门项目的高评分偏差，公式中每个评分减去对应项目的平均分 $R - iR - i$ 。

选择相似度方法需结合数据特性：若用户评分存在明显尺度差异（如严格型与宽容型用户），皮尔逊系数更优；若需快速处理高维稀疏数据，余弦法更为高效。

评分预测 确定目标用户 u 的最近邻集合 $N(u)$ （即相似度最高的 k 个用户）后，预测其对未评分项目 i 的兴趣分值。预测公式为加权平均：

$$\hat{R}_{u,i} = \bar{R}_u + \frac{\sum_{v \in N(u)} \sin(u, v) \cdot (R_{v,i} - \bar{R}_v)}{\sum_{v \in N(u)} |\sin(u, v)|}$$

- 此公式通过引入用户平均评分 R_u 、 R_v 消除个体评分偏差，并利用相似度作为权重，强调高相似用户的评分影响。最终，系统按预测分值降序推荐Top-N项目给用户。

User-Based CF的优势在于直观性强，能够发现长尾项目，但面临计算复杂度高（用户数远大于项目数）、冷启动（新用户数据稀疏）等挑战。其适用于用户规模相对稳定、用户行为数据丰富的场景（如电商、社交平台），尤其在隐式反馈场景中，通过行为日志构建评分矩阵，可有效捕捉用户偏好动态变化。

4.1.2 代码实现

数据预处理与矩阵构建 User-Based协同过滤的数据预处理与矩阵构建过程是推荐系统实现的核心基础。其核心目标是将原始评分数据转化为结构化矩阵表示，为后续的相似度计算和预测提供高效的数据支撑。整个处理流程可分为数据加载、特征工程和矩阵化转换三个阶段：

数据加载阶段采用双层字典结构进行原始数据存储。通过遍历训练文件，以用户行为为中心建立双向索引。特征工程阶段着重于数据标准化处理。针对每个用户的评分记录计算均值评分，该均值将用于后续相似度计算时的评分中心化处理（如皮尔逊相关系数需要扣除用户评分偏差）。此处的均值计算采用numpy向量化操作，避免循环带来的性能损耗：

```
1 self.mean_ratings = {
2     user: np.mean(list(ratings.values()))
3     for user, ratings in self.user_ratings.items()
4 }
```

矩阵化转换阶段构建用户-物品的二维评分矩阵。首先建立双向索引映射：将原始用户ID和物品ID分别映射为矩阵的行列索引，通过字典结构实现 $O(1)$ 复杂度的ID查找。随后初始化零值矩阵，遍历用户评分字典填充有效评分。这种稀疏矩阵表示方法既压缩了存储空间，又保持了矩阵运算的便利性：

```

1  # 建立索引映射
2  self.user_to_idx = {user: idx for idx, user in enumerate(users)}
3  self.item_to_idx = {item: idx for idx, item in enumerate(items)}
4
5  # 矩阵填充
6  for user, ratings in self.user_ratings.items():
7      user_idx = self.user_to_idx[user]
8      for item, rating in ratings.items():
9          if item in self.item_to_idx: # 过滤未出现在物品索引中的条目
10             item_idx = self.item_to_idx[item]
11             self.user_item_matrix[user_idx, item_idx] = rating

```

相似度计算 User-Based协同过滤的相似度计算本质上是将用户行为数据映射到向量空间，通过量化向量间几何关系来建立用户相似性度量。其数学核心在于构建用户评分向量并定义合适的距离函数，其中余弦相似度与皮尔逊相关系数是两种最具代表性的空间映射方法。

余弦相似度直接采用原始评分向量计算空间夹角余弦值，适用于显式评分数据且不考虑用户评分尺度差异的场景。代码实现借助sklearn的优化矩阵运算：

```

1  def cosine_similarity(matrix):
2      # 使用sklearn的cosine_similarity函数
3      return cosine_similarity(matrix)

```

皮尔逊相关系数则通过统计学视角改进相似度度量，其计算分为三个关键步骤：首先进行均值中心化消除用户评分偏差；随后通过标准差标准化将向量缩放到可比尺度；最终执行协方差计算。该方法的代码实现显式展现其数学本质：

```

1  def pearson_similarity(matrix):
2      mean = np.mean(matrix, axis=1, keepdims=True)
3      centered = matrix - mean
4      std = np.sqrt(np.sum(centered ** 2, axis=1, keepdims=True))
5      std[std == 0] = 1
6      normalized = centered / std
7      return np.dot(normalized, normalized.T)

```

两种方法均通过矩阵运算实现高效计算，其中皮尔逊方法通过中心化处理能更准确捕捉用户间相对评分模式，而余弦相似度保留原始评分绝对值差异。

冷启动处理 协同过滤系统在处理冷启动问题时，主要面临两种情况：新用户冷启动和新物品冷启动。在我们的实现中，通过引入全局平均评分（global mean rating）来优雅地处理这两种情况。首先，在模型训练阶段，我们计算并存储全局平均评分：

```

1  # 在fit方法中计算全局平均分
2  all_ratings = [] # 存储所有评分用于计算全局平均分
3  for user in self.user_ratings:
4      for item, score in self.user_ratings[user].items():
5          all_ratings.append(score)
6  self.global_mean_rating = np.mean(all_ratings) if all_ratings else 0

```

在预测阶段，当遇到新用户或新物品时，系统会进行冷启动检查。这个检查过程非常关键，它需要同时验证用户和物品是否在训练集中：

```

1  def predict(self, user_id, item_id):
2      # 冷启动处理：如果用户或物品不在训练集中，返回全局平均分
3      if item_id not in self.item_to_idx or user_id not in self.user_to_idx:
4          return self.global_mean_rating

```

这种处理方式的优势在于：全局平均评分能够反映整个评分系统的整体水平，它比单独使用用户平均分或物品平均分更加合理。在预测过程中，如果找不到足够的相似用户或相似物品，系统也会使用全局平均分作为默认值。

模型预测与评估 算法首先在用户相似度矩阵中定位目标用户的近邻集合，该集合需满足双重约束：**相似度超过预设阈值且对目标物品有历史评分记录**。通过相似度排序截取Top-N邻居后，**执行偏差修正的加权平均计算——每个邻居的贡献由其与目标用户的相似度加权**，同时扣除该邻居的平均评分偏差以消除个体评分尺度差异。具体实现如代码所示：

```
1 # 偏差调整计算核心逻辑
2 numerator = 0
3 denominator = 0
4 for other_user_id, similarity in similar_users:
5     rating = self.user_ratings[other_user_id][item_id]
6     numerator += similarity * (rating - self.mean_ratings[other_user_id]) # 偏差调整项
7     denominator += abs(similarity) # 相似度正则化
8
9 predicted_rating = self.mean_ratings[user_id] + numerator / denominator
```

评估 User-Based CF 模型的性能通常使用验证集来进行。主要的评估指标包括 **平均绝对误差**（Mean Absolute Error, MAE）和 **均方根误差**（Root Mean Square Error, RMSE）。这些指标能够量化模型预测评分与真实评分之间的差异。

4.2 Item-Based CF

Item-Based协同过滤（Item-Based CF）的核心假设是**相似物品可能获得同一用户的相近评分**。与User-Based CF的用户相似性驱动不同，其以物品为分析主体，通过挖掘物品间的共现评分模式构建推荐模型。输入数据同样基于用户-物品评分矩阵，但建模焦点转向物品维度，构建**物品-用户评分矩阵**，矩阵元素 $R_{i,u}$ 表示用户 u 对物品 i 的评分，未评分项作为预测目标。

4.2.1 算法原理

物品相似性度量聚焦于共同评分用户的偏好一致性，计算复用4.1.1节的通用方法，分为**余弦相似度**，直接计算物品评分向量的夹角余弦；**皮尔逊相关系数**，消除用户评分偏差，将评分中心化处理。相似度计算后形成物品相似度矩阵，用于后续邻居选择。

预测用户 u 对目标物品 i 的评分时，执行两个步骤：1.**相似物品筛选**：从物品相似度矩阵中提取与 i 最相似的 k 个物品（阈值过滤后），构成邻居集合 $N(i)$ ；2.**加权评分聚合**：基于用户 u 对 $N(i)$ 中物品的历史评分，计算加权预测值：

$$\hat{R}_{u,i} = \bar{R}_i + \frac{\sum_{j \in N(i)} \sin(i,j) \cdot (R_{u,j} - \bar{R}_j)}{\sum |\sin(i,j)|}$$

其中， $\bar{R}_i$ 为物品 i 的平均评分，通过引入物品平均分消除热门物品的评分偏差。最终评分通过截断处理约束在合理范围（如0-100）。**冷启动处理策略**与User-Based CF对称：若目标物品或用户未出现在训练集中，直接返回全局平均分；若用户未对任何相似物品评分，则依赖物品平均分或全局均值填补。

Item-Based CF适用于**物品数量相对稳定、用户行为动态性强**的场景（如新闻推荐、短视频流），其优势包括：

- **计算效率高**：物品数通常远小于用户数，相似度矩阵规模更小；
- **实时性优**：物品相似度可离线预计算，在线预测仅需局部加权；
- **隐式反馈适配**：通过用户行为频次构建物品关联，增强可解释性。

与User-Based CF形成互补，二者可根据业务特性选择或融合，以平衡推荐多样性、时效性与计算开销。

4.2.2 代码实现



注意: ItemCF在代码实现上和UserCF大抵相似, 鉴于篇幅, 下述只保留部分关键代码, 在后续算法中, 同样省略冗余代码

在数据加载与预处理层面, ItemCF首先从训练文件中加载用户-物品评分数据, 构建物品-用户评分矩阵。这一步与UserCF类似, 但ItemCF更关注物品之间的相似性, 构建的是**物品-用户评分矩阵**。在相似度计算方面, 与UserCF不同, ItemCF使用**物品-用户评分矩阵**来计算物品相似度。

```
1 # 计算物品相似度矩阵
2 if self.similarity_method == 'cosine':
3     self.item_similarity = cosine_similarity(self.item_user_matrix)
4 elif self.similarity_method == 'pearson':
5     self.item_similarity = pearson_similarity(self.item_user_matrix)
6 else:
7     raise ValueError(f"不支持的相似度计算方法: {self.similarity_method}")
```

ItemCF通过**用户已评分的物品**来预测未评分物品的评分。具体来说, 对于目标物品, 找到与之最相似的物品, 并根据用户对这些相似物品的评分来预测目标物品的评分。

```
1 def predict(self, user_id, item_id):
2     # 冷启动处理: 如果用户或物品不在训练集中, 返回全局平均分
3     if item_id not in self.item_to_idx or user_id not in self.user_to_idx:
4         return self.global_mean_rating
5
6     item_idx = self.item_to_idx[item_id]
7     user_idx = self.user_to_idx[user_id]
8
9     # 获取物品的相似物品
10    similar_items = []
11    for other_item_idx, similarity in
12    enumerate(self.item_similarity[item_idx]):
13        if other_item_idx != item_idx and similarity > self.min_similarity:
14            other_item_id = self.idx_to_item[other_item_idx]
15            if other_item_id in self.user_ratings[user_id]:
16                similar_items.append((other_item_id, similarity))
17
18    # 按相似度排序并选择top-N个邻居
19    similar_items.sort(key=lambda x: x[1], reverse=True)
20    similar_items = similar_items[:self.n_neighbors]
21
22    # 计算预测评分
23    numerator = 0
24    denominator = 0
25    for other_item_id, similarity in similar_items:
26        rating = self.user_ratings[user_id][other_item_id]
27        numerator += similarity * (rating -
28    self.mean_ratings[other_item_id])
29        denominator += abs(similarity)
30
31    predicted_rating = self.mean_ratings[item_idx] + numerator / denominator
```

4.3 Slope one

Slope One算法是一种简单而高效的协同过滤推荐算法，由Daniel Lemire和Anna Maclachlan于2005年首次提出。该算法的核心思想基于一个重要的观察：**不同用户对同一对商品的评分差异往往保持相对稳定**。换句话说，如果用户A对商品i的评分比对商品j的评分高2分，那么其他用户对这两个商品的评分差异也很可能接近2分。**这种基于评分差异的线性关系**假设使得Slope One算法能够通过计算商品间的平均评分差异来进行有效的评分预测，而无需进行复杂的矩阵分解或相似度计算。

4.3.1 算法原理

Slope One算法的核心在于计算和维护商品间的平均评分差异。对于任意两个商品i和j，算法首先计算所有同时对这两个商品进行过评分的用户的评分差异，然后求取这些差异的平均值作为商品i和j之间的偏差值。数学上，商品i相对于商品j的平均偏差可以表示为：

$$dev(i, j) = \Sigma(u \in S(i, j))(r_{ui} - r_{uj}) / |S(i, j)|$$

- 其中，S(i,j)表示同时对商品i和j进行过评分的用户集合， r_{ui} 和 r_{uj} 分别表示用户u对商品i和j的评分， $|S(i, j)|$ 表示集合S(i,j)的大小。这个偏差值反映了用户群体对商品i相对于商品j的整体偏好程度。

基于计算得到的商品间偏差值，Slope One算法可以预测用户u对未评分商品i的评分。预测公式采用加权平均的方式，考虑用户u已评分的所有商品与目标商品i之间的偏差关系：

$$\hat{r}_{ui} = (\Sigma(j \in R_u)(r_{uj} + dev(i, j)) \times |S(i, j)|) / (\Sigma(j \in R_u)|S(i, j)|)$$

- 其中， R_u 表示用户u已经评分过的商品集合。在这个预测公式中，每个已评分商品j对预测结果的贡献由两部分组成：用户u对商品j的实际评分 r_{uj} ，以及商品i相对于商品j的平均偏差 $dev(i, j)$ 。权重 $|S(i, j)|$ 表示计算偏差值 $dev(i, j)$ 时使用的样本数量，样本数量越多，对应的偏差值越可靠，因此在预测中应该给予更高的权重。

与传统的协同过滤算法相比，Slope One算法具有计算简单、易于理解和实现的特点，同时在许多实际应用场景中能够取得令人满意的推荐效果。该算法特别适用于用户评分数据相对稠密且评分差异模式较为稳定的推荐系统。

4.3.2 代码实现

Slope One算法的实现过程相对简单直观，主要分为两个阶段：预计算阶段和预测阶段。在预计算阶段，**算法需要遍历所有的用户评分数据，计算每对商品之间的平均偏差值**。这个过程的时间复杂度为 $O(n^2m)$ ，其中n是商品数量，m是用户数量。虽然这个复杂度看似较高，但由于预计算只需要进行一次，且结果可以持久化存储，因此在实际应用中是可以接受的。

```

1     def fit(self, trainset):
2
3         cdef int n_items = trainset.n_items
4
5         cdef long[:, ::1] freq = np.zeros((trainset.n_items,
6 trainset.n_items), np.int_)
7         cdef double[:, ::1] dev = np.zeros((trainset.n_items,
8 trainset.n_items), np.double)
9         cdef int u, i, j, r_ui, r_uj
10
11         AlgoBase.fit(self, trainset)
12
13         for u, u_ratings in trainset.ur.items():

```



```

12         for i, r_ui in u_ratings:
13             for j, r_uj in u_ratings:
14                 freq[i, j] += 1
15                 dev[i, j] += r_ui - r_uj
16
17         for i in range(n_items):
18             dev[i, i] = 0
19             for j in range(i + 1, n_items):
20                 dev[i, j] /= freq[i, j]
21                 dev[j, i] = -dev[i, j]
22
23         self.freq = np.asarray(freq)
24         self.dev = np.asarray(dev)
25
26         self.user_mean = [np.mean([r for (_, r) in trainset.ur[u]])
27                             for u in trainset.all_users()]
28
29         return self

```

在预测阶段，当需要为用户 u 预测对商品 i 的评分时，算法会查找用户 u 已评分的所有商品，然后利用这些商品与目标商品 i 之间的预计算偏差值来生成预测评分。这个过程的时间复杂度为 $O(|R_u|)$ ，其中 $|R_u|$ 是用户 u 已评分的商品数量。由于大多数用户的评分商品数量相对有限，因此预测过程通常很快。

```

1     def estimate(self, u, i):
2         if not (self.trainset.knows_user(u) and
3                 self.trainset.knows_item(i)):
4             raise PredictionImpossible('User and/or item is unknown.')
5
6         Ri = [j for (j, _) in self.trainset.ur[u] if self.freq[i, j] > 0]
7         est = self.user_mean[u]
8         if Ri:
9             est += sum(self.dev[i, j] for j in Ri) / len(Ri)
10
11         return est

```

5. 图模型与集成方法

5.1 Graph CF

基于图的协同过滤将推荐问题转化为图上的相似度传播问题，通过随机游走计算用户和物品之间的相似度，并利用相似度和统计信息进行评分预测。这种方法能够捕获用户-物品交互的复杂结构，从而提供准确的推荐。

5.1.1 算法原理

算法将用户-物品交互关系建模为二部图（bipartite graph），通过图上的相似性传播实现推荐。其核心思想是将用户和物品视为图节点，评分行为视为边，利用图结构捕捉高阶相似性。以下是算法的核心原理：

图构建 首先，我们构建一个二部图 $G=(U \cup I, E)$ ，其中 U 是用户节点集合， I 是物品节点集合， E 是连接用户和物品的边的集合。每条边 (u, i) 表示用户 u 对物品 i 有过评分，边的权重通常与评分值相关。在此次实现中，权重被设计为归一化后的评分的绝对值，并乘以一个与用户和物品评分标准差相关的因子（以考虑评分分布的差异）。具体而言，权重计算如下：

$$\text{weight}(u, i) = |r_{ui}| \times (1 + 0.1 \times (\sigma_u + \sigma_i))$$

- 其中 r_{ui} 是归一化后的评分， σ_u 和 σ_i 分别是用户 u 和物品 i 的评分标准差。这样设计权重可以使得评分分布较广的用户或物品对相似度传播有更大的影响。

转移概率矩阵 构建一个转移概率矩阵 P 来描述图中节点间的转移概率。矩阵 P 的大小为 $(|U| + |I|) \times (|U| + |I|)$ 。对于每个用户节点 u ，其指向物品节点 i 的转移概率为：

$$P_{u \rightarrow i} = \frac{\text{weight}(u, i)}{\sum_{j \in N(u)} \text{weight}(u, j)}$$

- 其中 $N(u)$ 是用户 u 评分过的物品集合。同样，从物品节点 i 指向用户节点 u 的转移概率也类似，此处省略。

随机游走 随机游走过程用于计算节点之间的相似度。这里采用带重启的随机游走（Random Walk with Restart, RWR），其思想是：从一个节点出发，以概率 α 按照转移概率矩阵进行游走，以概率 $1-\alpha$ 跳回起始节点（重启）。通过多次迭代，可以计算一个节点到其他节点的访问概率，这些概率可以解释为节点间的相似度。迭代公式为：

$$S^{(t+1)} = \alpha \cdot S^{(t)} \cdot P + (1 - \alpha) \cdot I$$

- 其中 $S^{(t)}$ 是第 t 步的相似度矩阵， I 是单位矩阵（表示重启时回到自身）。初始时 $S^{(0)}=I$ 。经过 n 次迭代后，得到的 $S^{(n)}$ 即为最终的相似度矩阵。这个相似度矩阵 `scores` 记录了任意两个节点之间的相似度。

评分预测 对于给定的用户 u 和物品 i ，我们利用随机游走得到的相似度分数 `score` 进行预测。预测公式综合考虑了用户平均评分、物品平均评分以及相似度分数：

$$\hat{r}_{ui} = \mu_u + \beta \cdot \text{score}_{u,i} \cdot \sigma_u + \gamma \cdot (\mu_i - \mu_{\text{global}})$$

- 其中 μ_u 是用户 u 的平均评分， σ_u 是用户 u 的评分标准差， μ_i 是物品 i 的平均评分， μ_{global} 是全局平均评分。在代码中，系数 β 取0.7， γ 取0.3。最后，将预测评分限制在评分范围内（如0到100）。

冷启动处理：复用4.1.1节的全局平均分策略

基于图的协同过滤能够捕捉用户和物品之间的高阶关系（通过多步游走），而传统的协同过滤（如基于邻域的方法）通常只考虑直接邻居。该方法对稀疏数据有较好的鲁棒性，因为随机游走能够通过多次跳转探索更广泛的连接。需要注意的是，该算法的计算复杂度主要在于转移概率矩阵的构建和随机游走的迭代，当用户和物品数量很大时，矩阵可能非常大，迭代计算会消耗较多内存和时间。

5.1.2 代码实现

基于图的协同过滤算法的代码实现主要细节如下：

首先，在 `_normalize_ratings` 中，算法先汇集所有评分计算全局均值，然后针对每个用户与每件物品分别计算均值与标准差，保证评分分布信息被保留又能消除量纲影响。这样用 Z-score 归一化后，不会因为某个用户或物品评分方差过小而导致数值爆炸。


```

1 all_ratings = [r for ur in self.user_ratings.values() for r in ur.values()]
2 self.global_mean = np.mean(all_ratings) if all_ratings else 0
3 for user_id, ratings in self.user_ratings.items():
4     self.user_mean_ratings[user_id] = np.mean(ratings.values())
5     self.user_std[user_id] = max(np.std(ratings.values()), 1.0)
6 # 归一化
7 normalized = (rating - user_mean) / user_std

```

接着在 `_build_graph` 中，构造一个无向二部图，用户和物品分别作为两类节点，通过编号映射 `user_map`、`item_map` 与反向映射保证能在后续矩阵中定位。添加边时，以归一化评分绝对值加权，再略微放大用户与物品评分波动对权重的影响，这种设计既体现了评分强度，又兼顾了评分的离散程度：

```

1 weight = abs(rating) * (1 + 0.1 * (self.user_std[user_id] +
2 self.item_std[item_id]))
3 self.G.add_edge(f"u_{user_id}", f"i_{item_id}", weight=weight)

```

随后在 `_build_transition_matrix` 中，算法在大小为 `(n_users + n_items)` 的方阵中填充转移概率。对每个用户到其评分物品的边，按权重占比归一化，并对称地将概率赋给物品到用户：

```

1 total = sum(weights)
2 self.P[u_idx, i_idx] = weight / total
3 self.P[i_idx, u_idx] = weight / total

```

核心的随机游走在 `_random_walk` 中迭代完成，融合了重启机制与传播机制，在每次迭代更新中既保留了原始意义的身份矩阵，又不断吸纳网络中新的评分关联：

```

1 scores = np.eye(n)
2 for _ in range(self.n_iter):
3     scores = self.alpha * scores.dot(self.P) + (1 - self.alpha) * np.eye(n)

```

经过 `n_iter` 次迭代后，`scores[u, i]` 即衡量了用户与物品间的关联强度。

最后在 `predict` 方法中，当要预测某用户对某物品的评分时，先从 `scores` 中读取二者的随机游走得分，再结合用户自身评分分布与物品相对全局的偏移做线性融合，并裁剪到 `[min_rating, max_rating]` 范围：

```

1 score = self.scores[u_idx, i_idx]
2 pred = user_mean + score * user_std * 0.7 + (item_mean - self.global_mean) *
3 0.3
4 pred = max(self.min_rating, min(self.max_rating, pred))

```

这种融合充分利用了全局、用户和物品三个层面的统计信息，既有图结构挖掘的相似度，又兼顾了用户与物品的固有评分倾向。

5.2 TopKNanCF

TopKNanCF是一种创新的混合推荐算法，将物品协同过滤与 梯度提升树 (GBDT) 模型相结合。其核心思想是**为每个目标物品独立训练预测模型，使用其最相似的物品作为特征输入**。这种方法通过机器学习模型捕捉用户评分与相似物品间的复杂非线性关系，并能智能处理特征缺失问题。

5.2.1 算法原理

TopKNaiveCF基于一个关键想法：用户对目标物品的评分模式，可以通过该用户对相似物品的评分组合来精确预测。它为每个物品独立构建预测模型，充分利用物品间的局部关联性。

物品相似度计算 采用余弦相似度衡量物品间的关联强度。

特征工程与缺失处理 对于每个目标物品，算法选取其Top-K最相似物品作为特征集。当预测用户u对目标物品i的评分时，特征向量定义为：

$$X_u = [R_{u,s_1}, R_{u,s_2}, \dots, R_{u,s_K}] \quad (s_k \in \text{TopK}(i))$$

关键创新在于允许特征缺失 - 当用户未对某相似物品评分时，特征值设为 `NaN`。GBDT模型能自动处理这种缺失，在节点分裂时优化缺失值路径，无需人工填充。

梯度提升树模型 对每个目标物品独立训练GBDT回归模型：

$$\hat{y} = \sum_{m=1}^M \gamma_m h_m(x)$$

- h_m 为决策树， γ_m 为学习率。每棵树拟合当前残差（前序模型预测与真实值之差），通过迭代降低预测误差。算法选用 `HistGradientBoostingRegressor` 实现，该变种采用直方图算法加速训练，支持大规模数据。

5.2.2 代码实现

数据加载与物品相似度矩阵构建 首先遍历训练字典，构建用户→物品→评分的三层嵌套结构，并在收集评分时计算全局平均分作为冷启动预测值，以便高效查询用户历史行为。接着以物品为中心构建物品→用户的倒排索引，大幅提升物品相似度计算效率，尤其在物品数远小于用户数时更为显著。在相似度计算阶段，通过双重循环遍历所有物品对，先找出共同评分用户并提取其评分向量，再以余弦相似度（当分母为零时设相似度为0）计算相似度，最终将结果存储为以物品ID为键、相似度字典为值的嵌套字典形式。

GBDT模型训练 核心是为每个物品训练专属预测模型：

```
1  for target_item in items:
2      # 获取Top-K相似物品
3      sim_items = sorted(sim_matrix.get(target_item, {}).items(), key=lambda
x: x[1], reverse=True)
4      topk_items = [item for item, _ in sim_items[: self.topk]]
5      self.topk_dict[target_item] = topk_items
```

首先确定目标物品的Top-K相似物品。相似物品按相似度降序排序后取前K个，存储在 `topk_dict` 中供后续预测使用。这种局部关联性建模是算法高效的关键。

```
1  X = []; y = []
2  for user in users:
3      x = []
4      for item in topk_items:
5          # 关键：允许特征缺失(NaN)
6          x.append(user_ratings[item] if item in user_ratings else np.nan)
7      X.append(x)
8      y.append(user_ratings[target_item])
```

特征矩阵 `X` 的构建是算法精髓。对于目标物品的每个评分用户，遍历其Top-K相似物品：若用户评过则取实际评分，否则设为 `NaN`。标签 `y` 为用户对目标物品的实际评分。这种设计使模型能够学习在部分信息缺失情况下的评分模式。

```
1 model = HistGradientBoostingRegressor(max_iter=100, random_state=42)
2 model.fit(X, y)
3 self.models[target_item] = model
```

使用 `HistGradientBoostingRegressor` 训练GBDT模型。该实现通过直方图近似加速训练，默认100次迭代平衡精度与效率。`random_state` 确保可复现性。模型存储在字典中，键为目标物品ID，值为训练好的GBDT模型。

预测流程与冷启动 多层回退机制确保预测鲁棒性。冷启动场景同样分三种情况处理：目标物品无训练模型（新物品），用户不在训练集（新用户），物品有模型但用户无历史记录，算法优先尝试使用用户历史平均分，若无历史记录则回退到全局平均分。

特征构建逻辑与训练时完全一致：遍历目标物品的Top-K相似物品，若用户评过则取值，否则置为 `NaN`。向量重塑为二维数组以满足模型输入格式。

```
1 try:
2     pred = self.models[item_id].predict(x)[0]
3 except Exception:
4     pred = np.mean(list(user_ratings.values())) if user_ratings else
    self.global_mean
```

调用目标物品的GBDT模型预测评分。异常捕获机制防止预测失败：当模型异常时（如输入格式问题），回退到用户平均分或全局平均分。

评估机制 标准指标量化模型性能，评估函数遍历验证字典中的每个用户-物品评分记录，调用预测函数得到预测值，最终计算MAE和RMSE。

6. 线性优化模型

6.1 GDLinearCF

GDLinearCF的核心思想是利用目标物品的相似物品评分作为特征，通过线性模型学习用户评分模式。这种方法跟多关注物品间的关联性，并通过机器学习模型捕捉这些关联与最终评分之间的复杂关系。整个算法流程围绕三个核心阶段展开：物品相似度计算、特征工程构建和梯度下降模型训练。

6.1.1 算法原理

模型的核心假设是：用户对目标物品的评分可以由其相似物品的评分线性表示。这个假设基于物品间的内在关联性，比如用户如果喜欢某个导演的电影，很可能也会喜欢该导演的其他作品。整个算法流程分为三个关键阶段：

相似度计算 同样采用余弦相似度衡量物品间的相似性，每个物品仅保留相似度最高的K个邻居，形成相似物品字典 `topk_dict`。这种设计确保了计算效率，同时聚焦于最相关的物品关联。

特征工程 复用以下特征处理方法：缺失值填充：物品平均分 $\rightarrow$ 全局平均分；特征标准化： $(x - \mu) / \sigma$

模型训练 使用带L2正则化的线性回归模型：

$$\hat{y} = w^T x + b$$

损失函数设计为均方误差加正则项：

$$\mathcal{L} = \frac{1}{N} \sum (y - \hat{y})^2 + \frac{\lambda}{2} \|w\|^2$$

通过梯度下降优化参数，并引入学习率指数衰减策略：

$$\eta_t = \eta_0 \times 0.95^t$$

这种设计既防止过拟合，又确保训练后期参数更新更加精细，提升模型收敛稳定性。

6.1.2 代码实现

数据加载与预处理 在数据加载阶段，算法接受字典形式的训练数据，其中每个用户对应其评分物品的字典。通过双层循环，算法构建了 `user_ratings` 字典（用户为键，物品评分字典为值），同时收集所有评分计算全局平均分。这个全局平均分在后续处理中扮演重要角色，**既作为特征的一部分，也是处理缺失值的默认值。**

物品相似度计算 首先构建物品-用户倒排索引，之后对于每对物品，先找到共同评分的用户集合，使用余弦相似度计算充分利用NumPy的优化函数，特别处理了分母为零的边界情况。

特征向量构建 特征构建是模型的核心创新点。对于每个用户-物品评分记录，算法先获取目标物品的Top-K相似物品列表。然后检查用户是否对这些相似物品评过分：若有则采用实际评分，否则使用物品平均分或全局平均分智能填充。随后添加三个关键统计特征：用户历史平均分（反映用户评分习惯）、目标物品平均分（反映物品受欢迎程度）、全局平均分（提供基准参考）。

```
1  for user, items in self.user_ratings.items():
2      user_mean = np.mean(list(items.values())) if items else self.global_mean
3      for target_item, target_score in items.items():
4          features = []
5          topk_items = self.topk_dict.get(target_item, [])
6          for sim_item in topk_items:
7              rating = items.get(sim_item) # 用户是否评分过该相似物品
8              if rating is None:
9                  rating = self.item_mean.get(sim_item, self.global_mean)
10             features.append(rating)
11
12         if len(features) < self.topk:
13             features += [self.global_mean] * (self.topk - len(features))
14         features.append(user_mean)
15         features.append(self.item_mean.get(target_item, self.global_mean))
16         features.append(self.global_mean)
17         X.append(features)
18         y.append(target_score)
```

归一化处理 算法分别计算每个特征的均值和标准差，然后进行标准化：(特征值 - 均值)/标准差。特别值得注意的是对标准差为零的特征的处理——将其标准差设为1，避免除零错误。标签同样进行标准化，使模型更容易学习。

梯度下降训练 训练过程采用小批量梯度下降优化。每次迭代中，先计算当前学习率（指数衰减策略确保后期更新更精细）。然后进行前向传播计算预测值，并与真实值比较得到误差。梯度计算包含两部分：数据误差导数和L2正则化项。梯度裁剪技术将梯度限制在[-1,1]范围内，有效防止梯度爆炸问题。参数更新采用标准的梯度下降规则，学习率随迭代次数衰减使训练更稳定。

```
1  w = np.random.randn(n_features) * 0.01 # 小随机数初始化
2  b = 0.0 # 偏置项初始化
3
4  for epoch in range(self.epochs):
5
6      current_lr = self.lr * (0.95 ** epoch) # 学习率指数衰减
7      y_pred = X_normalized @ w + b # 前向传播：计算预测值
8      error = y_pred - y_normalized # 计算误差
```

```

9
10     grad_w = (X_normalized.T @ error) / n_samples + self.reg_lambda * w
11     grad_b = np.mean(error)
12     grad_w = np.clip(grad_w, -1.0, 1.0)
13     grad_b = np.clip(grad_b, -1.0, 1.0)
14     w -= current_lr * grad_w
15     b -= current_lr * grad_b
16
17     self.w = w
18     self.b = b

```

预测流程 复用训练时的特征工程逻辑，确保一致性。对于给定的用户-物品对，算法首先获取用户历史评分，然后构建与训练时完全相同的特征向量：包括相似物品评分、用户平均分、物品平均分和全局平均分。特征标准化使用训练阶段保存的参数，确保相同处理方式。

```

1  user_history = self.user_ratings.get(user_id, {})
2  user_avg = np.mean(list(user_history.values())) if user_history else
   self.global_mean
3  features = []
4
5  similar_items = self.topk_dict.get(item_id, [])[:self.topk]
6
7  for sim_item in similar_items:
8      if sim_item in user_history:
9          features.append(user_history[sim_item])
10     else:
11         features.append(self.item_mean.get(sim_item, self.global_mean))
12
13  features += [self.global_mean] * (self.topk - len(features))
14  features.append(user_avg)
15  features.append(self.item_mean.get(item_id, self.global_mean))
16  features.append(self.global_mean)
17
18  features = np.array(features)
19  features_std = (features - self.X_mean) / self.X_std
20  pred_normalized = features_std @ self.w + self.b
21  pred = pred_normalized * self.y_std + self.y_mean
22
23  return max(0, min(100, pred))

```

模型评估 评估函数遍历验证字典中的每个用户-物品评分记录，调用预测函数得到预测评分，并与真实评分一起存储。最后计算两个关键指标MAE和RMSE。

6.2 LeastSquares CF

最小二乘协同过滤的核心思想基于一个核心假设：**每个物品的评分模式可以通过其他相关物品的评分线性表示**。这种线性依赖关系源于用户行为模式的稳定性——如果用户对一组特征物品的评分模式与目标物品存在稳定的数学关系，那么这种关系可以通过线性回归模型精确捕捉。算法本质上是为每个目标物品*i*构建定制化的预测模型，通过最小二乘法求解最优权重系数，直接从用户-物品评分数据中提取物品间的内在关联。

6.2.1 算法原理

特征物品选择策略 在为特定目标物品*i*构建模型前，算法首先识别与其关联性最强的特征物品集合 I_k 。这种关联性不是基于内容相似性，而是纯粹基于用户行为模式的统计相关性。具体而言，选择那些与目标物品*i*共同被评分频率最高的*k*个物品（即 `top_n` 参数控制的特征物品数量）。这一过程可以形式化为：

$$\text{feature_items} = \underset{j \neq i}{\operatorname{argtopk}} \left(\sum_{u \in U_i} \mathbf{1}_{(u,j) \in R} \right)$$

- 其中 U_i 表示所有对物品*i*评分的用户集合， $\mathbf{1}$ 是指示函数。这种方法确保特征物品与目标物品具有强行为关联性，为后续建模提供可靠基础。选择共同评分频率最高的物品而非相似度最高的物品，使模型能更直接地捕捉用户行为模式中的关联性。

线性回归模型构建 确定了特征物品集后，算法建立线性回归模型描述目标物品与特征物品的关系：

$$R_{u,i} = \sum_{j \in I_k} w_j \cdot R_{u,j} + \epsilon$$

- 其中， $R_{u,i}$ 表示用户*u*对目标物品*i*的评分， I_k 是与*i*相关的*k*个特征物品集合， w_j 为特征物品*j*的线性权重， $R_{u,j}$ 是用户*u*对特征物品*j*的评分， ϵ 为残差项。

通过最小二乘法求解权重向量 w ，最小化所有评分用户的残差平方和：

$$\min_w \sum_{u \in U_i} \left(R_{u,i} - \sum_{j \in I_k} w_j \cdot R_{u,j} \right)^2$$

- U_i 表示对目标物品*i*有评分的用户集合， w 是待优化的权重向量。

缺失值处理机制 缺失值处理复用7.1.1节的全局平均分填充策略。

评分预测 预测用户*u*对物品*i*的评分时，算法执行两阶段计算：首先获取用户*u*对特征物品集 I_k 的评分（缺失值用 R_{global} 填充），然后将这些特征评分与训练所得权重*w*进行线性组合：

$$\hat{R}_{u,i} = \sum_{j \in I_k} w_j \cdot \hat{R}_{u,j}$$

最小二乘协同过滤在物品间存在显式依赖关系的场景中表现突出，如系列电影、同品牌商品或技术规格相似的产品推荐。通过为每个物品定制线性组合模型，它能精确捕捉物品间的关联模式，相比传统邻域方法具有更强的解释性——权重系数直接量化特征物品对目标物品的影响程度。然而，该方法也面临计算复杂度随`top_n`增大而增加的问题，且对新物品存在冷启动挑战（缺乏历史评分无法构建模型）。模型效果还依赖于用户对特征物品的覆盖率，当用户评分记录过少时预测可靠性降低。

6.2.2 代码实现

数据准备与索引构建 算法初始阶段构建全局评分基准和高效的双向查询结构。通过遍历训练字典，同时建立用户->物品的正向索引和物品->用户的倒排索引，为后续特征选择提供即时访问能力。全局平均分的计算为缺失值填补建立中立基准。这种双向索引设计使算法能快速响应两种关键查询："用户评过哪些物品"和"物品被哪些用户评过分"，大幅提升后续特征选择的效率。全局平均分作为数据中性基准，为缺失值填补提供合理依据。

特征物品选择过程 针对每个目标物品，算法直接分析其评分用户群体的行为模式，统计这些用户评分的其他物品的共现频率。通过Python的Counter类高效完成频次统计，并选择最高频的*k*个物品作为特征集：


```

1  other_items_counter = Counter()
2  # 遍历目标物品的所有评分用户
3  for user in item_user[target_item].keys():
4      # 收集该用户除目标物品外的所有评分物品
5      other_items = [item for item in self.user_ratings[user] if item !=
6                      target_item]
7      other_items_counter.update(other_items) # 更新共现频次
8
9  # 选取共现频率最高的top_n个物品
10 most_common_items = other_items_counter.most_common(self.top_n)
11 feature_items = [item for item, _ in most_common_items] # 提取物品ID

```

这种方法从行为数据中直接挖掘“评过目标物品的用户还常评哪些物品”的关联模式，比传统相似度计算更直观高效。特征物品选择完全基于实际用户行为，确保模型建立在真实的关联模式上。

模型训练与权重求解 对于每个目标物品，算法构建特征矩阵X和目标向量y。特征矩阵的每行对应一个用户对特征物品的评分（缺失值用全局平均分填补），目标向量则为这些用户对目标物品的真实评分：

```

1  X, y = [], []
2  for user in item_user[target_item].keys():
3      # 构建特征向量：用户对特征物品的评分（缺失时用全局均值填补）
4      features = [self.user_ratings[user].get(item, self.global_mean)
5                  for item in feature_items]
6      X.append(features)
7      y.append(self.user_ratings[user][target_item]) # 目标物品真实评分
8
9  # 使用NumPy的最小二乘求解器（自动处理秩亏问题）
10 weights, _, _, _ = np.linalg.lstsq(np.array(X), np.array(y), rcond=None)
11 self.item_weights[target_item] = (feature_items, weights) # 存储模型

```

关键在于 `self.user_ratings[user].get(item, self.global_mean)` 调用，它优雅地处理了用户未评分特征物品的情况。`rcond=None` 参数使求解器自动调整对病态矩阵的处理策略，确保数值稳定性。每个物品的权重向量独立存储，实现完全个性化的预测模型。

分层预测策略 预测阶段根据物品和用户状态采用不同策略：对已训练物品使用定制线性模型；对新物品但老用户采用用户平均分；全新实体采用全局平均分：

```

1  def predict(self, user_id, item_id):
2      # 新物品处理：回退到用户平均分或全局平均分
3      if item_id not in self.item_weights:
4          user_ratings = self.user_ratings.get(user_id, {})
5          return np.mean(list(user_ratings.values())) if user_ratings else
6              self.global_mean
7
8      # 已有模型的物品：获取特征物品列表和权重向量
9      feature_items, weights = self.item_weights[item_id]
10     user_ratings = self.user_ratings.get(user_id, {})
11
12     # 动态构建特征向量（缺失值用全局均值填补）
13     features = [user_ratings.get(item, self.global_mean) for item in
14                 feature_items]
15
16     # 点积计算预测分并约束范围
17     pred = np.dot(weights, features)
18     return max(0, min(100, pred)) # 强制在0-100分范围内

```


这种分层设计确保在各种边界情况下都有合理预测：`item_id not in self.item_weights` 处理新物品冷启动，`user_ratings.get(user_id, {})` 处理新用户。最后的截断操作维护评分系统的边界一致性。

流式评估与指标计算 评估过程采用增量计算方式，仅需存储当前预测误差的累加值，无需缓存整个验证集的预测结果。这种流式处理使内存占用保持恒定，不受验证集规模影响。平方根运算在累加完成后执行一次，避免重复计算开销。

7. 矩阵分解模型

7.1 SVD

SVD（奇异值分解）算法是一种**矩阵分解技术**，通过将用户-物品评分矩阵分解为低维的用户特征向量和物品特征向量的乘积，来预测缺失评分并捕捉潜在偏好关系。

7.1.1 算法原理

其核心是将原始评分矩阵 R （维度 $m \times n$ ，其中 m 为用户数， n 为物品数）分解为两个低维矩阵的乘积：用户特征矩阵 P （维度 $m \times K$ ）和物品特征矩阵 Q （维度 $n \times K$ ），即 $R \approx PQT$ 。这里的 K 是潜在特征维度（ $K \ll \min(m, n)$ ），用于降维和捕捉用户与物品的隐含特征（如用户偏好、物品特性）。

损失函数 在传统的SVD矩阵分解方法中，算法的核心目标是通过最小化预测评分与实际评分之间的平方误差来学习用户和商品的潜在特征向量。具体而言，SVD算法的损失函数可以表示为：

$$L = \sum_{(u,i) \in R} (r_{ui} - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$$

这个损失函数的第一项代表了预测误差的平方和，其中 r_{ui} 是用户 u 对商品 i 的实际评分， $p_u^T q_i$ 是通过矩阵分解得到的预测评分。第二项是L2正则化项，通过引入正则化参数 λ 来控制模型复杂度，防止过拟合现象的发生。正则化项的作用在于约束用户特征向量 p_u 和商品特征向量 q_i 的范数，使得模型能够更好地泛化到未见过的数据上。

	i_1	i_2	...	i_j	...	i_n
u_1						
u_2						
...						
u_i						
...						
u_n						

然而，实际应用中，损失函数常扩展为包含偏差项，以更好地建模评分偏差：

$$L = \sum_{(u,i) \in R} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

- 其中 μ 是全局平均评分， b_u 是用户偏差（表示用户评分习惯）， b_i 是物品偏差（表示物品受欢迎程度）。这能更准确地捕捉评分中的系统性偏差。 **

正则化SVD引入 在稀疏数据条件下，观测到的评分集合 R 相对较小，容易导致传统SVD模型出现过拟合。为了解决这一问题，需要在损失函数中引入正则化项，使模型在拟合训练数据的同时保持适当的复杂度。基于此思路，正则化SVD（RSVD）的完整目标函数可以写成：

$$\min_{p_u, q_i} \sum_{(u,i) \in R} \left(r_{ui} - \sum_{k=1}^K p_{u,k} q_{k,i} \right)^2 + \frac{\lambda}{2} \sum_u \|p_u\|^2 + \frac{\lambda}{2} \sum_i \|q_i\|^2$$

- 其中 $\lambda > 0$ 是正则化参数，用于平衡模型拟合程度与复杂度。
- 通过加上 $\frac{\lambda}{2} \sum_u \|p_u\|^2 + \frac{\lambda}{2} \sum_i \|q_i\|^2$ ，RSVD 在保证拟合效果的同时，降低了参数过大导致的过拟合风险。

优化方法 对于上述目标函数，常见的求解方法包括：**迭代最小二乘（ALS）**：交替固定 $\{p_u\}$ 或 $\{q_i\}$ 进行优化，直至收敛。虽然原理清晰，但对于大规模数据集而言，实现较为繁琐、效率不够高。**随机梯度下降（SGD）**：针对每个训练样本 (u,i) ，计算预测误差，然后沿梯度反方向更新参数。

7.1.2 代码实现

SVD算法是推荐系统中最著名的矩阵分解方法，在Netflix Prize竞赛中由Simon Funk推广而闻名。该算法的核心预测公式基于用户和物品的偏置以及潜在因子的内积计算。当不使用偏置时，算法等价于概率矩阵分解方法。

```
1  class SVD(AlgoBase):
2
3      def __init__(self, n_factors=100, n_epochs=20, biased=True, init_mean=0,
4                      init_std_dev=.1, lr_all=.005,
5                      reg_all=.02, lr_bu=None, lr_bi=None, lr_pu=None,
6                      lr_qi=None,
7                      reg_bu=None, reg_bi=None, reg_pu=None, reg_qi=None,
8                      random_state=None, verbose=False):
9
10         self.n_factors = n_factors
11         self.n_epochs = n_epochs
12         self.biased = biased
13         self.init_mean = init_mean
14         self.init_std_dev = init_std_dev
15         self.lr_bu = lr_bu if lr_bu is not None else lr_all
16         self.lr_bi = lr_bi if lr_bi is not None else lr_all
17         self.lr_pu = lr_pu if lr_pu is not None else lr_all
18         self.lr_qi = lr_qi if lr_qi is not None else lr_all
19         self.reg_bu = reg_bu if reg_bu is not None else reg_all
20         self.reg_bi = reg_bi if reg_bi is not None else reg_all
21         self.reg_pu = reg_pu if reg_pu is not None else reg_all
22         self.reg_qi = reg_qi if reg_qi is not None else reg_all
23         self.random_state = random_state
24         self.verbose = verbose
25
26     AlgoBase.__init__(self)
```

在代码实现中，算法通过随机梯度下降(SGD)来最小化正则化平方误差。关键的参数配置包括`n_factors`用于设定潜在因子数量（默认100），`n_epochs`控制SGD迭代次数（默认20次）。算法支持`biased`参数来决定是否使用偏置项，当设为`False`时可获得无偏版本。

```

1     def fit(self, trainset):
2
3         AlgoBase.fit(self, trainset)
4         self.sgd(trainset)
5
6         return self

```

用户和物品因子的初始化遵循正态分布，可通过`init_mean`和`init_std_dev`参数调节均值和标准差。学习率控制通过`lr_all`统一设置（默认0.005），或者分别为不同参数类型设置`lr_bu`、`lr_bi`、`lr_pu`、`lr_qi`。正则化参数同样支持统一设置`reg_all`（默认0.02）或分别配置。

训练完成后，算法会生成四个重要属性：`pu`表示用户因子矩阵（ $n_users \times n_factors$ ），`qi`表示物品因子矩阵（ $n_items \times n_factors$ ），`bu`和`bi`分别表示用户和物品偏置向量。

```

1     def estimate(self, u, i):
2         # Should we cythonize this as well?
3
4         known_user = self.trainset.knows_user(u)
5         known_item = self.trainset.knows_item(i)
6
7         if self.biased:
8             est = self.trainset.global_mean
9
10            if known_user:
11                est += self.bu[u]
12
13            if known_item:
14                est += self.bi[i]
15
16            if known_user and known_item:
17                est += np.dot(self.qi[i], self.pu[u])
18
19        else:
20            if known_user and known_item:
21                est = np.dot(self.qi[i], self.pu[u])
22            else:
23                raise PredictionImpossible('User and item are unknown.')
24
25        return est

```

7.2 SVD++

然而，传统的SVD方法存在一个显著的局限性，即它仅仅考虑了显式的评分信息，而忽略了用户在系统中产生的大量隐式反馈信息。在实际的推荐系统应用场景中，用户的隐式行为数据（如商品浏览记录、购买历史、点击行为等）往往比显式评分更加丰富和容易获取。基于这一观察，SVD++算法应运而生，它在传统SVD的基础上融合了隐式反馈信息，从而能够更全面地建模用户的偏好模式。

7.2.1 算法原理



基础分解框架复用5.1.1节的 $R \approx PQ^T$ 模型

预测评分公式 SVD++算法的核心创新在于将用户的隐式反馈信息整合到评分预测模型中。该算法认为，即使用户没有对某个商品给出明确的评分，但用户与该商品的交互行为仍然能够反映用户的潜在兴趣。基于这一思想，SVD++算法重新定义了评分预测公式：

$$\hat{r}_{ui} = \mu + b_u + b_i + \left(p_u + |I_u|^{-1/2} \sum_{j \in I_u} y_j \right)^T q_i$$

在这个公式中，预测评分不再仅仅依赖于用户和商品的基本特征向量，而是引入了多个重要的改进元素。首先， μ 代表了整个系统的全局平均评分，它反映了所有用户对所有商品的整体评价趋势。其次， b_u 和 b_i 分别表示用户 u 和商品 i 的偏置项，这些偏置项能够捕捉个体用户和商品相对于全局平均水平的系统性偏差。

更为重要的是，SVD++算法在用户特征向量 p_u 的基础上增加了隐式反馈项 $|I_u|^{-1/2} \sum_{j \in I_u} y_j$ 。这里， I_u 表示用户 u 有过交互行为的商品集合， y_j 是商品 j 对应的隐式反馈因子向量，而 $|I_u|^{-1/2}$ 是归一化因子，用于消除不同用户交互商品数量差异带来的影响。这种设计使得算法能够从用户的历史行为模式中推断出用户的潜在偏好，即使这些偏好没有通过显式评分表达出来。

损失函数与正则化 为了学习上述模型中的所有参数，需要在损失函数中同时对显式与隐式部分进行约束。完整的目标函数可以表示为：

$$L = \sum_{(u,i) \in R} \left[r_{ui} - \mu - b_u - b_i - \left(p_u + |I_u|^{-1/2} \sum_{j \in I_u} y_j \right)^T q_i \right]^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + \|y_j\|^2 + b_u^2 + b_i^2)$$

- 第一项为预测误差的平方和，涵盖了显式评分与融合了隐式反馈后的预测值之差。
- 第二项为正则化项，对显式特征向量 p_u , q_i 、隐式反馈因子 y_j 以及偏置 b_u , b_i 一并加以约束，防止模型过拟合。

随机梯度下降 SVD++ 通常采用随机梯度下降（SGD）来优化上述损失函数。对于每一条训练样本 (u,i) ，首先计算预测误差：

$$e_{ui} = r_{ui} - \hat{r}_{ui} = r_{ui} - \left[\mu + b_u + b_i + \left(p_u + |I_u|^{-1/2} \sum_{j \in I_u} y_j \right)^T q_i \right].$$

然后，根据梯度信息更新各项参数。随机梯度更新方式保证了在大规模数据集上具有较高的计算效率，同时能够有效避免陷入局部最优。

相比于传统SVD（或RSVD）只针对评分矩阵进行分解，SVD++ 由于要处理额外的隐式反馈信息，因此计算量有所增加，但这部分开销是合理且必要的：在实际推荐场景中，隐式反馈极大地提升了模型的预测精度和推荐质量，尤其在数据稀疏、长尾物品较多的情况下优势更加明显。

7.2.2 代码实现

SVD++算法是SVD的扩展版本，其创新之处在于考虑了隐式评分信息。该算法引入了新的物品因子 y_j 来捕获隐式评分，即用户对物品进行评分这一行为本身，而不考虑具体评分值。

```
1 class SVDpp(AlgoBase):
2
3     def __init__(self, n_factors=20, n_epochs=20, init_mean=0,
4                   init_std_dev=.1,
```

```

4         lr_all=.007, reg_all=.02, lr_bu=None, lr_bi=None,
        lr_pu=None,
5         lr_qi=None, lr_yj=None, reg_bu=None, reg_bi=None,
        reg_pu=None,
6         reg_qi=None, reg_yj=None, random_state=None, verbose=False,
7         cache_ratings=False):
8
9     self.n_factors = n_factors
10    self.n_epochs = n_epochs
11    self.init_mean = init_mean
12    self.init_std_dev = init_std_dev
13    self.lr_bu = lr_bu if lr_bu is not None else lr_all
14    self.lr_bi = lr_bi if lr_bi is not None else lr_all
15    self.lr_pu = lr_pu if lr_pu is not None else lr_all
16    self.lr_qi = lr_qi if lr_qi is not None else lr_all
17    self.lr_yj = lr_yj if lr_yj is not None else lr_all
18    self.reg_bu = reg_bu if reg_bu is not None else reg_all
19    self.reg_bi = reg_bi if reg_bi is not None else reg_all
20    self.reg_pu = reg_pu if reg_pu is not None else reg_all
21    self.reg_qi = reg_qi if reg_qi is not None else reg_all
22    self.reg_yj = reg_yj if reg_yj is not None else reg_all
23    self.random_state = random_state
24    self.verbose = verbose
25    self.cache_ratings = cache_ratings
26
27    AlgoBase.__init__(self)

```

在代码参数配置上，SVD++的n_factors默认值调整为20，学习率lr_all默认设为0.007。算法新增了cache_ratings参数来决定是否在训练时缓存评分数据，这能加速训练但会增加内存占用。与SVD相比，SVD++增加了lr_yj和reg_yj参数来控制隐式因子的学习率和正则化。

训练后的模型会额外生成yj属性，表示隐式物品因子矩阵（n_items × n_factors），这与显式的qi因子矩阵配合使用，提升了预测准确性。

```

1    def estimate(self, u, i):
2
3        est = self.trainset.global_mean
4
5        if self.trainset.knows_user(u):
6            est += self.bu[u]
7
8        if self.trainset.knows_item(i):
9            est += self.bi[i]
10
11        if self.trainset.knows_user(u) and self.trainset.knows_item(i):
12            Iu = len(self.trainset.ur[u]) # nb of items rated by u
13            u_impl_feedback = (sum(self.yj[j] for (j, _)
14                                in self.trainset.ur[u]) / np.sqrt(Iu))
15            est += np.dot(self.qi[i], self.pu[u] + u_impl_feedback)
16
17        return est

```

7.3 NMF

非负矩阵分解 (Non-Negative Matrix Factorization, NMF) 是一种重要的数据分析技术, 其核心思想是将大规模数据集分解为更小且具有实际意义的组成部分, 同时确保所有数值保持非负性。这种约束条件使得NMF在提取数据中 有用特征方面表现出色, 并且大大简化了数据的分析和处理过程。与传统的矩阵分解方法相比, NMF的非负性约束更符合许多实际应用场景的物理意义, 例如在图像处理中像素值不能为负, 在文本分析中词频不能为负等情况。

7.3.1 算法原理

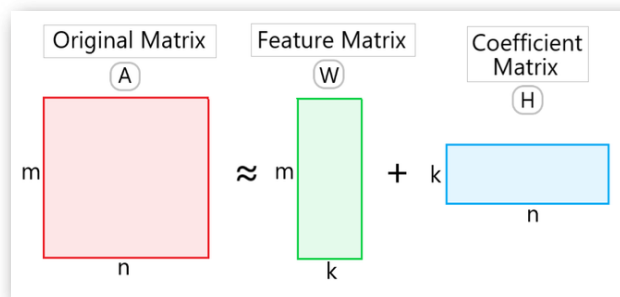


复用5.1.1节的矩阵分解框架 $R \approx PQ^T$

矩阵分解 对于一个维度为 $m \times n$ 的矩阵 A , 其中每个元素都大于等于0, NMF算法将其分解为两个矩阵 W 和 H , 这两个矩阵的维度分别为 $m \times k$ 和 $k \times n$, 且两个矩阵都只包含非负元素。这种分解关系可以用数学公式表示为:

$$A \approx WH, \quad W \in \mathbb{R}_{\geq 0}^{m \times k}, \quad H \in \mathbb{R}_{\geq 0}^{k \times n}, \quad k \leq \min(m, n).$$

这里, k 称为分解秩, 通常取值远小于 $\min(m, n)$, 以实现降维和去噪的效果。矩阵 W 的每一列可以看作若干“基础模式”或“特征向量”, 这些向量共同组成原始数据中存在的潜在结构; 而 H 的每一列则记录了对应原始列向量 (如某条样本或某个文档) 在这些基础模式下的权重。由于两者都仅允许非负元素, 因此分解结果可被直观解释为“将原始数据用若干个非负基础部分按加权叠加的方式重构”。



重构误差 要使 WH 尽可能接近 A , 通常采用最小化 Frobenius 范数的重构误差:

$$\min_{W \geq 0, H \geq 0} \|A - WH\|_F^2 = \min_{W \geq 0, H \geq 0} \sum_{i=1}^m \sum_{j=1}^n (A_{ij} - [WH]_{ij})^2,$$

并要求 $W_{il} \geq 0, H_{lj} \geq 0$ 。由于这个优化问题是非凸的, 通常只能找到局部最优解。在实际求解过程中, 首先需要为 W 和 H 赋予非负的初始值, 常见做法包括从均匀分布或正态分布 (取绝对值) 中采样, 或者基于截断 SVD 等启发式方法进行初始化。良好的初始化有助于加速收敛并减少陷入较差局部最优的概率。

迭代更新 随后进入迭代更新阶段, 以逐步减少 $\|A - WH\|$ 。这里最经典的优化技术是**乘法更新规则**:

$$W_{il} \leftarrow W_{il} \times \frac{[AH^T]_{il}}{[WHH^T]_{il}}, \quad H_{lj} \leftarrow H_{lj} \times \frac{[W^T A]_{lj}}{[W^T WH]_{lj}}.$$

每次更新后, W 与 H 都保持非负, 并且 $\|A - WH\|$ 在迭代过程中单调不增。通常交替执行若干次上述更新, 直至满足如下任一停止条件: 重构误差变化幅度低于预设阈值、达到最大迭代次数, 或其他用户自定义的收敛标准。

另一个常用的方法是**交替最小二乘法 (ALS)**。其思路是固定 H 后, 将 $\min_{W \geq 0} \|A - WH\|_F^2$ 转化为“带非负约束的最小二乘”子问题, 通过合适的 NNLS 方法求得最优 W ; 接着固定新得到的 W , 以同样方式求解 $\min_{H \geq 0} \|A - WH\|_F^2$ 子问题获得 H , 然后再回到第一步循环。交替迭代直到收敛。ALS 在处理大规模、稀疏数据时往往比乘法更新收敛更快, 但需要调用 NNLS 求解器。

应用场景 从直觉与应用角度来看, NMF 的关键在于“以非负并且可解释的方式, 将复杂数据拆解为若干有意义的部件再加权组合”。这使得它特别适合的场景为: 人脸图像分解 文本主题建模 音频信号分离。

正是因为 NMF 保持了“非负”“部分叠加”的直观含义，并能够揭示出数据中隐藏的局部结构，它在图像、文本、音频等多个领域都得到了广泛且有效的应用。

7.3.2 代码实现

NMF算法基于非负矩阵分解理论，确保所有用户和物品因子保持非负值。该算法的预测公式与SVD相似，但通过特殊的梯度下降步长选择来维持因子的非负性约束。

```
1  class NMF(AlgoBase):
2
3      def __init__(self, n_factors=15, n_epochs=50, biased=False, reg_pu=.06,
4                      reg_qi=.06, reg_bu=.02, reg_bi=.02, lr_bu=.005, lr_bi=.005,
5                      init_low=0, init_high=1, random_state=None, verbose=False):
6
7          self.n_factors = n_factors
8          self.n_epochs = n_epochs
9          self.biased = biased
10         self.reg_pu = reg_pu
11         self.reg_qi = reg_qi
12         self.lr_bu = lr_bu
13         self.lr_bi = lr_bi
14         self.reg_bu = reg_bu
15         self.reg_bi = reg_bi
16         self.init_low = init_low
17         self.init_high = init_high
18         self.random_state = random_state
19         self.verbose = verbose
20
21         if self.init_low < 0:
22             raise ValueError('init_low should be greater than zero')
23
24         AlgoBase.__init__(self)
```

在参数设置上，NMF的n_factors默认值较小（15），但n_epochs增加到50次以确保充分收敛。算法默认不使用偏置（biased=False），但可以启用偏置版本来提高准确性，尽管这可能增加过拟合风险。

NMF的关键配置包括reg_pu和reg_qi（默认0.06）来控制用户和物品因子的正则化，以及init_low和init_high（默认0到1）来设置因子初始化范围。由于算法对初始值高度敏感，这两个参数的调节需要格外谨慎。

```
1      def estimate(self, u, i):
2          # Should we cythonize this as well?
3          known_user = self.trainset.knows_user(u)
4          known_item = self.trainset.knows_item(i)
5
6          if self.biased:
7              est = self.trainset.global_mean
8              if known_user:
9                  est += self.bu[u]
10             if known_item:
11                 est += self.bi[i]
12             if known_user and known_item:
13                 est += np.dot(self.qi[i], self.pu[u])
14
```

```

15         else:
16             if known_user and known_item:
17                 est = np.dot(self.qi[i], self.pu[u])
18             else:
19                 raise PredictionImpossible('User and item are unknown.')
20
21     return est

```

这三种算法都支持random_state参数来控制随机数生成，确保结果的可重现性，并提供verbose参数来显示训练进度。在实际应用中，开发者需要根据具体数据特征和性能要求来选择合适的算法和参数配置，以获得最佳的推荐效果。

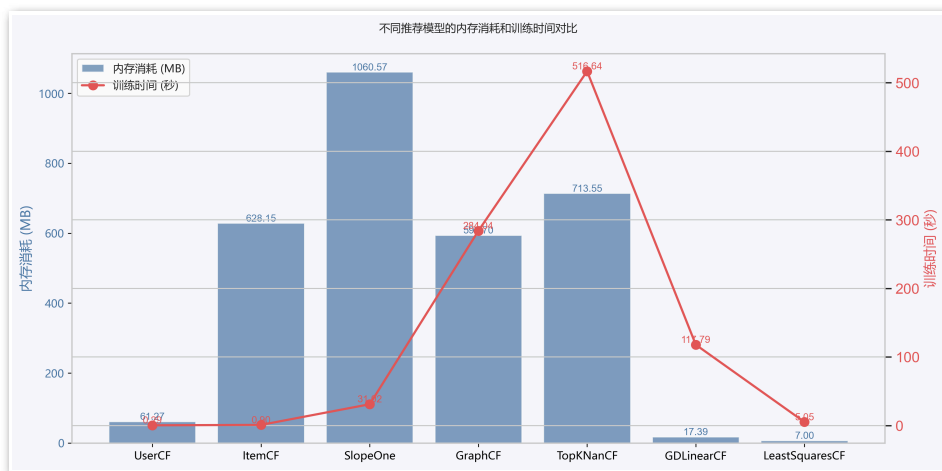
8. 实验结果



在代码编写过程中，本小组采用了两种不同的代码框架来构建模型，鉴于保证实验结果的有效性和公平性，部分实验分析的图表分成两个"7+3"模式，即SVD、SVD++、NMF分成一组，其他算法分成一组，从而在相同环境下比较差距。

8.1 时间与内存消耗分析

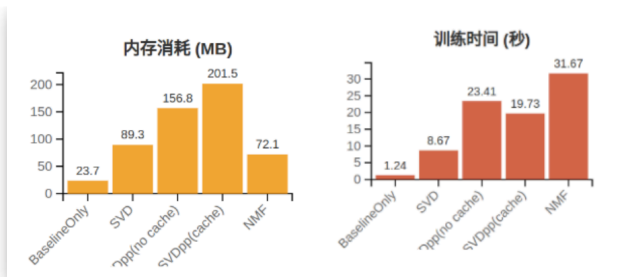
统一的数据集和参数配置下，对比了UserCF、ItemCF、SlopeOne、GraphCF、TopKNaNCF、GDLinearCF、LeastSquaresCF的性能表现。实验过程中，所有模型均采用相同的训练集、验证集划分（2:8比例），并详细记录每个模型在训练阶段的内存消耗（MB）和训练耗时（秒）。模型参数方面，邻居数量（n_neighbors）统一设为20，相似度计算方法主要采用余弦相似度（cosine），其余参数均为各算法推荐的默认配置，最大程度保证了实验的公平性和复现性。结果如下图表：



从实验结果来看，UserCF、LeastSquaresCF等模型在内存消耗和训练时间上表现最为轻量，能够在较短时间内完成模型训练，且对内存资源的占用极低，适合对资源敏感或需要快速响应的应用场景。相比之下，SlopeOne、GraphCF和TopKNaNCF等模型由于在训练过程中需要进行大规模的矩阵运算、图结构构建或集成学习，导致内存消耗和训练时间显著增加。ItemCF模型的内存消耗也相对较大，但训练时间依然保持在较低水平，说明其在物品相似度计算和离线处理方面具有一定的工程优势。

整体来看，模型的内存和时间消耗与其算法复杂度和实现机制密切相关。轻量级模型适合对实时性和资源消耗有严格要求的场景，而高复杂度模型虽然在资源消耗上存在劣势，但在特定任务中可能具备更强的表达能力和泛化能力。因此，在实际应用中需要根据业务需求、数据规模和硬件资源合理选择推荐算法，以实现性能与效率的最佳平衡。

另一组代码框架下的内存和时间对比如下图表：



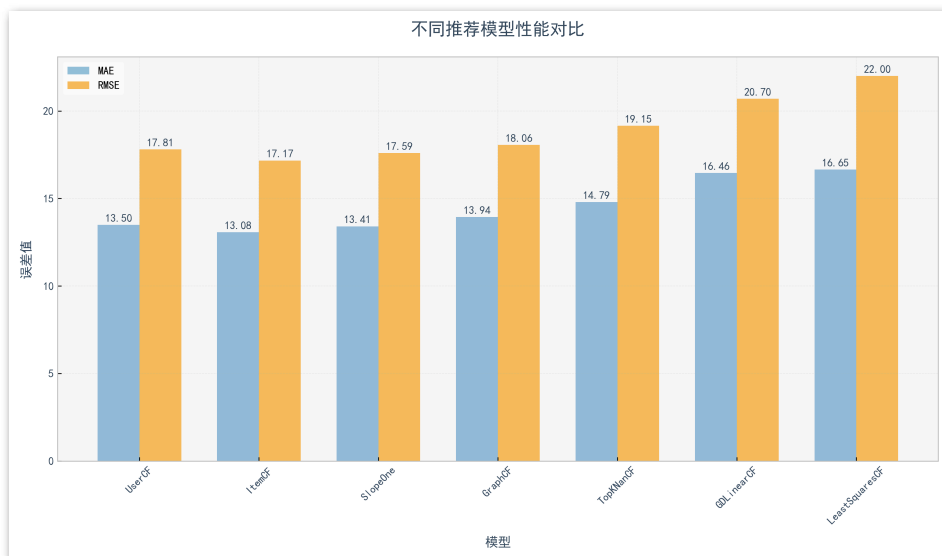
BaselineOnly方法由于模型结构最为简单，无需复杂的参数存储和矩阵分解，因此无论在内存消耗还是训练时间上都远低于其他方法。SVD方法引入了矩阵分解，模型参数量有所增加，导致内存消耗上升至89.3MB，训练时间也提升到8.67秒，但整体仍处于较低水平。

SVDpp方法由于考虑了用户的隐式反馈，模型复杂度进一步提升。未使用缓存时，SVDpp的内存消耗达到156.8MB，训练时间为23.41秒，均高于SVD。引入缓存机制后，SVDpp(cache)的内存消耗进一步增加至201.5MB，但训练时间反而有所下降，仅为19.73秒。这表明缓存机制虽然带来了更高的内存开销，但有效减少了重复计算，从而提升了训练效率。

NMF方法在本次实验中表现出较高的训练时间（31.67秒），为所有方法中最高，说明其在大规模数据下的计算复杂度较高。不过其内存消耗为72.1MB，低于SVDpp系列，显示其参数存储相对更为紧凑。

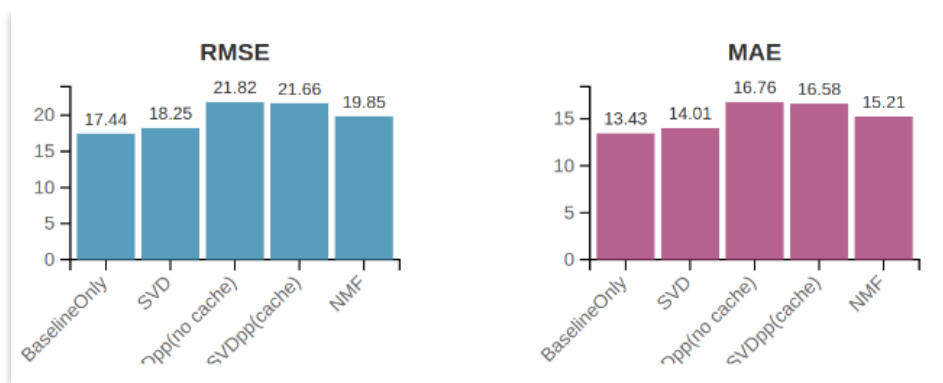
8.2 全局误差对比

统一的数据集和参数配置下，对比七种协同过滤推荐算法的性能表现，评估指标选用平均绝对误差（MAE）和均方根误差（RMSE）。结果如下图表：



从实验结果来看，ItemCF模型在MAE和RMSE两个指标上均取得了最优表现，分别为13.08和17.17，显示出较强的评分预测能力。SlopeOne和UserCF的表现也较为接近，MAE和RMSE均处于较低水平，说明传统的基于邻域的协同过滤方法在数据稠密度较高时依然具有较强的竞争力。

GraphCF模型通过引入图结构和随机游走机制，能够捕捉用户与物品之间的高阶关系，但在本实验中其误差略高于传统CF方法。TopKNNCF、GDLInearCF和LeastSquaresCF等集成与线性优化方法的误差相对较高，尤其是LeastSquaresCF的RMSE达到22.00，显示出在当前数据分布下，简单线性模型难以充分挖掘用户-物品间的复杂关联，这与本数据集物品数量远大于用户数量且评分极度稀疏有极大关系。



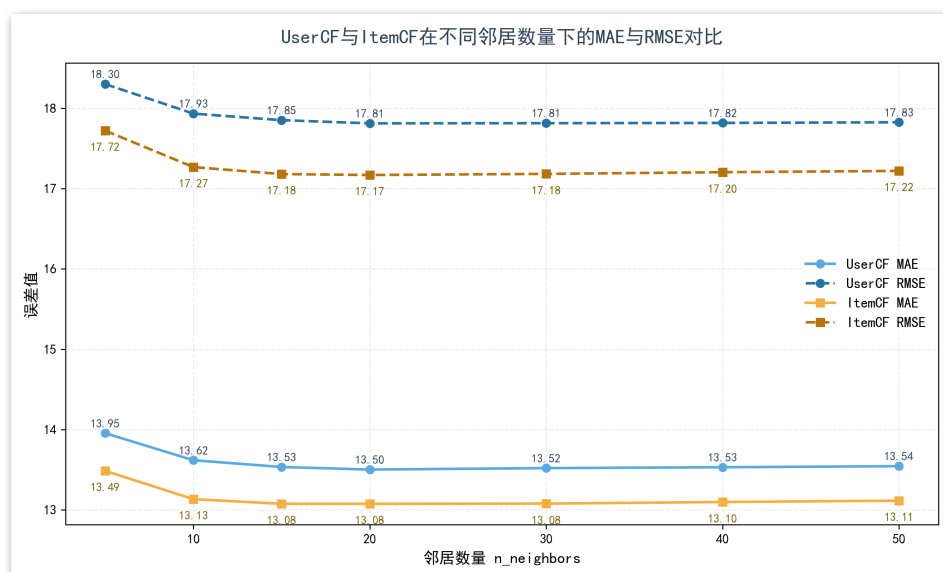
从RMSE和MAE的实验结果来看，BaselineOnly方法在两项指标上均取得了最低的误差，分别为17.44和13.43，说明在当前数据集和参数设置下，简单的全局均值预测反而具有较强的稳健性。SVD方法引入了矩阵分解，理论上能够更好地捕捉用户和物品的潜在特征，但实际误差略有上升，RMSE为18.25，MAE为14.01，表明其泛化能力受限于数据稀疏或参数未充分优化。

SVDpp方法在未使用缓存和使用缓存的情况下，RMSE和MAE均高于SVD，显示其在本实验条件下未能有效提升预测精度，可能与隐式反馈信息噪声较大或模型复杂度过高有关。NMF方法的表现介于SVD和SVDpp之间，RMSE为19.85，MAE为15.21，说明其在分解方式和特征表达上具有一定优势，但仍未超越最简单的BaselineOnly。整体来看，复杂模型在本实验中未能带来误差的显著降低，后续需从数据质量或参数调优等方面进一步优化。

8.3 模型特性分析

8.3.1 基于不同邻居数量下的误差对比

在邻居数量从5递增至50，其他参数如相似度计算方法均设为cosine等相同情况下，系统性地对比了UserCF与ItemCF两种协同过滤算法在不同邻居数量（n_neighbors）下的MAE与RMSE表现。结果如下表：



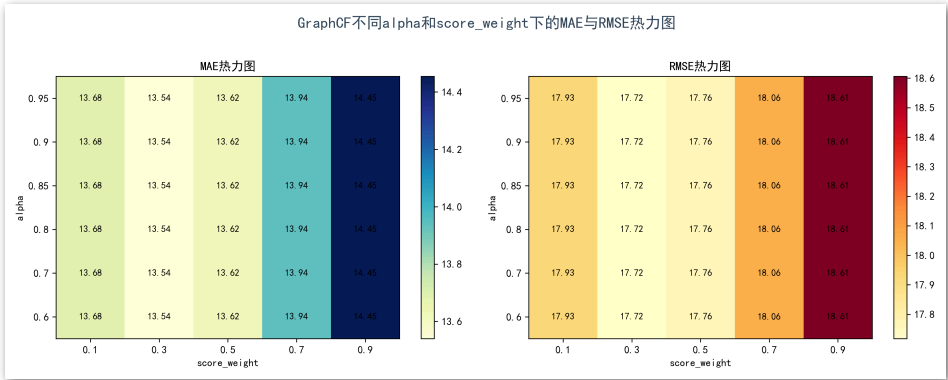
从结果曲线可以看出，随着邻居数量的增加，两种算法的误差指标均呈现出先下降后趋于平稳的趋势，且ItemCF在各个邻居数量下的MAE和RMSE均优于UserCF，表现出更强的评分预测能力。

值得注意的是，当邻居数量超过一定阈值后，误差曲线趋于平稳，进一步增大邻居数量对模型性能提升已无明显作用，这一现象可能与数据稀疏性有关——在邻居数量较大时，部分用户或物品已无足够高相似度的邻居可选，模型实际采用了全局或均值填充策略，导致误差收敛于一个稳定区间。

8.3.2 基于GraphCF的不同参数分析

此部分针对GraphCF模型的两个关键参数——随机游走重启概率alpha和评分融合权重score_weight，进行系统的自动调参分析。**alpha**参数用于控制随机游走过程中分数在原节点和邻居节点之间的分配比例，理论上能够影响信息在图结构中的传播深度和范围；而**score_weight**则决定了最终评分预测中图结构传播分数与物品均值偏移的融合比例，从而调节模型对结构信息与统计信息的依赖程度。

使用相同数据集，采用alpha取值范围为[0.6, 0.7, 0.8, 0.85, 0.9, 0.95]，score_weight取值范围为[0.1, 0.3, 0.5, 0.7, 0.9]，每组参数下均训练20次迭代并评估MAE与RMSE。结果如热力图所示：



从 结果可以看出，**alpha**的变化对模型误差几乎没有影响，说明在当前实现和数据分布下，分数传播的深度和重启概率对最终预测结果影响有限；而**score_weight**的变化则带来了明显的误差波动，**随着score_weight增大，模型对图结构分数的依赖增强，MAE和RMSE均呈现先降低后升高的趋势（0.3左右达到最低），表明合理融合结构信息与统计信息能够有效提升模型性能。**